

Week 10 lec 1

Function overriding

Method Overriding in C++

- If we inherit a class into the derived class and provide a definition for one of the base class's function again inside the derived class, then that function is said to be **overridden**, and this mechanism is called **Function Overriding**

Requirements for Overriding a Function

1. Inheritance should be there. Function overriding cannot be done within a class. For this we require a derived class and a base class.
2. Function that is redefined must have exactly the same declaration in both base and derived class, that means same name, same return type and same parameter list.

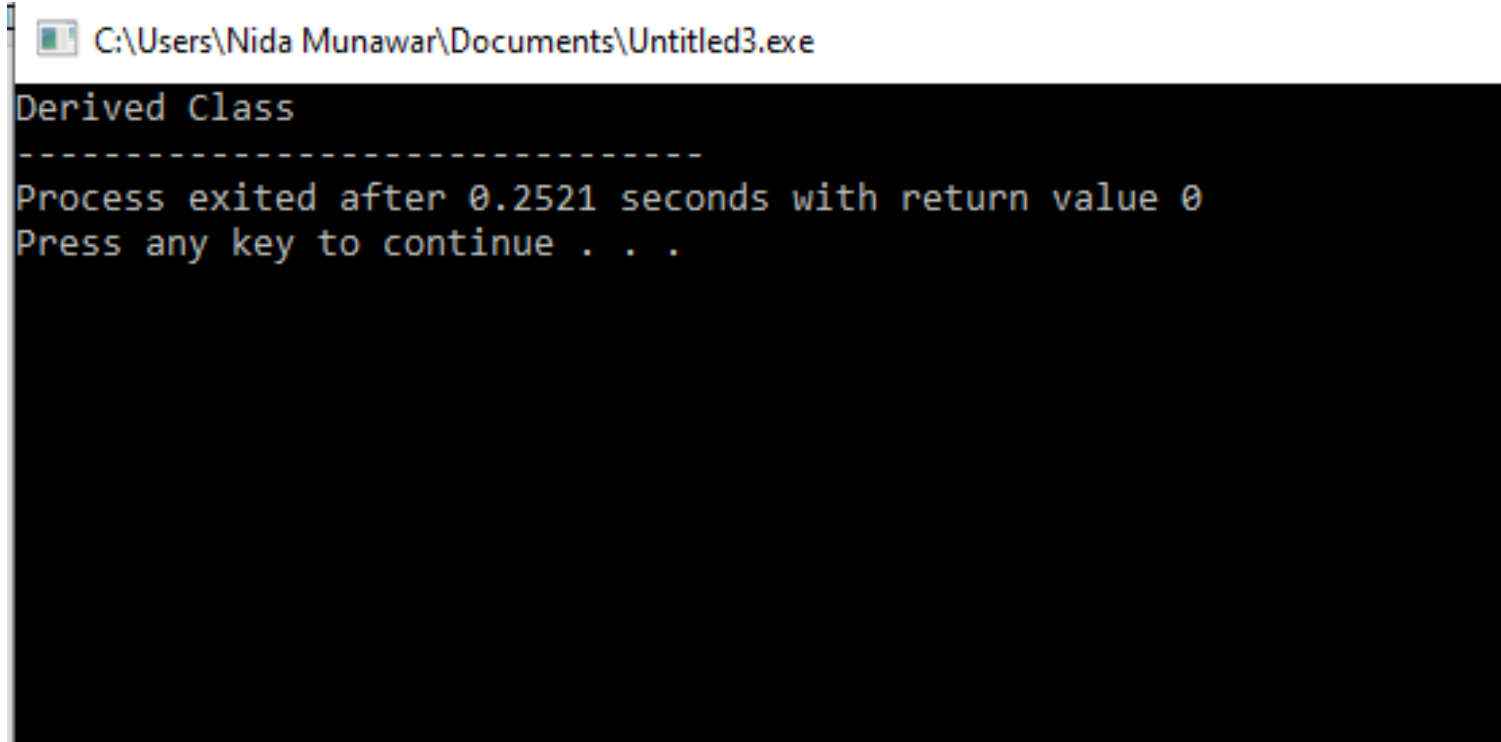
Early binding

```
class A {  
    public:  
    void display() {  
        cout<<"Base class"; } };  
class B:public A {  
    public:  
    int display() {  
        cout<<"Derived Class";  
        } };  
int main() {  
    B obj;  
    obj.display();  
    return 0; }
```

Early binding

- In the above example, we are calling the overridden function using derived class object. Compiler gives preference to object type.
- Base class object will call base version of the function and derived class's object will call the derived version of the function. This is called early binding
- In early binding return type and storage classes are ignored

Early binding



```
C:\Users\Nida Munawar\Documents\Untitled3.exe  
Derived Class  
-----  
Process exited after 0.2521 seconds with return value 0  
Press any key to continue . . .
```

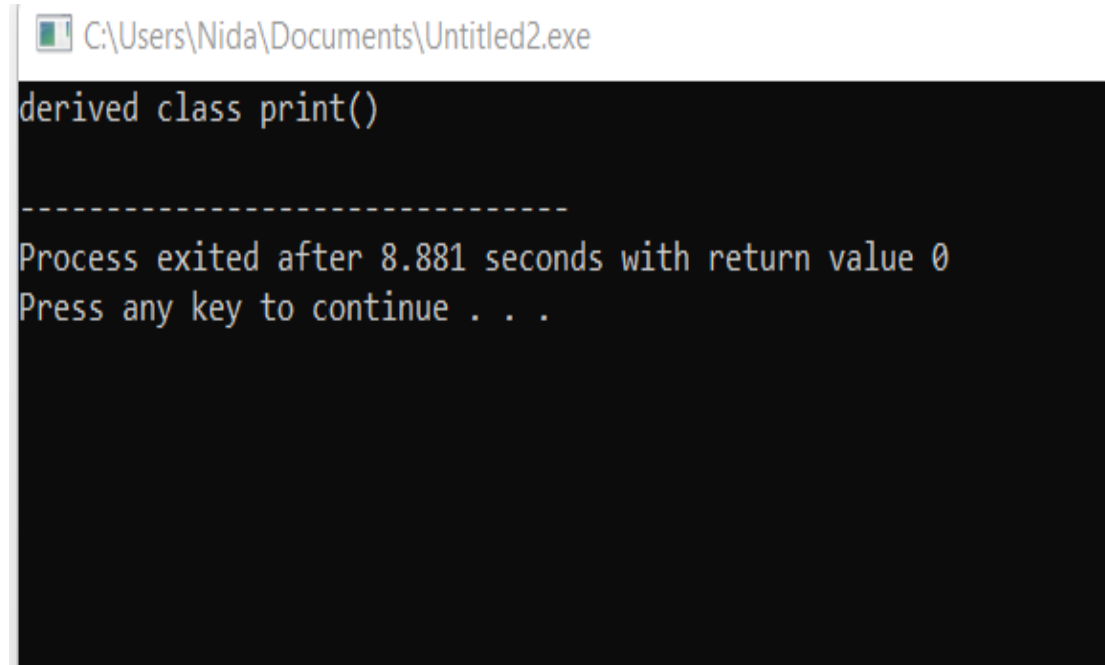
Function Call Binding with class Objects

- Connecting the function call to the function body is called **Binding**. When it is done before the program is run, its called **Early** Binding or **Static** Binding or **Compile-time** Binding.

Return type and storage class is not same

- class Base {
- public:
- void print() {
- cout << "base class print()" << endl; };
- class Derived : public Base {
- public:
- int print() {
- cout << "derived class print()" << endl;
- return 1;}
- };
- int main() {
- Derived derived1;
- derived1.print();}

In early binding return type and storage classes are ignored



```
C:\Users\Nida\Documents\Untitled2.exe
derived class print()
-----
Process exited after 8.881 seconds with return value 0
Press any key to continue . . .
```

Can we overload methods in sub class?

- Can we overload methods in sub class?
- Yes overloading can be done in different scopes but....

Can we overload methods in sub class?

C++ handles overloading across subclasses in a much different way. A member function declaration in a derived class does not overload member functions with the same name declared in the base class.

Rather, a member function in a derived class hides the declaration of the member functions with the same name in base class, even though the function parameter lists are different. Hence, to have member functions from a base class overload member functions from the derived class, the designer of the derived class can introduce the base class member functions into the scope of the derived class with using declarations.

Can we overload methods in sub class?

```
class Check{
    int i;
    public:
        Check(): i(0) { }
    void Display()
        { cout << "i of base " << i << endl; };
class Go : public Check{
    int i;
    public:
        Go(): i(1) { }
    void Display(int i)
        { cout << "i of child " << i << endl; };
```

Can we overload methods in sub class?

```
int main(){  
    Go g;  
    g.Display(3); // derived class will hide the Display() func of base  
class  
    return 0;}
```



C:\Users\Nida\Documents\Untitled1.exe

i of child 3

Process exited after 5.719 seconds with return value
Press any key to continue . . .

```
int main(){  
    Go g;  
    g.Display();  
    return 0;}
```

Can we overload methods in sub class?

Compiler (5) Resources Compile Log Debug Find Results Close			
Line	Col	File	Message
		C:\Users\Nida\Documents\Untitled1.cpp	In function 'int main()':
17	12	C:\Users\Nida\Documents\Untitled1.cpp	[Error] no matching function for call to 'Go::Display()'
17	12	C:\Users\Nida\Documents\Untitled1.cpp	[Note] candidate is:
13	2	C:\Users\Nida\Documents\Untitled1.cpp	[Note] int Go::Display(int)
13	2	C:\Users\Nida\Documents\Untitled1.cpp	[Note] candidate expects 1 argument, 0 provided

overloading methods in sub class

```
#include <iostream>
using namespace std;
class Base{
public:
    void foo(int i) {
        cout << "Inside foo(int i) i= " << i << "\n"; }
class Derived : public Base{
public:
    // using Base::foo; must be added
    // in absence of this simply foo(float f) is called
    // in its presence foo(int i) is called, no ambiguity
    Base::foo;
    void foo(float f) {
        cout << "Inside foo(float f) f= " << f << "\n"; }
};
```

overloading methods in sub class

```
int main(){  
    float f = 1.2f;  
    int i = 100;  
    Derived obj;  
    obj.foo(i); //no compiler error base foo  
    obj.foo(f); //no compiler error derived foo  
}
```

 C:\Users\Nida\Documents\Untitled1.exe

```
Inside foo(int i) i= 100
```

```
Inside foo(float f) f= 1.2
```

```
-----
```

```
Process exited after 1.286 seconds with return value 0
```

```
Press any key to continue . . .
```

overloading methods in sub class

overloading methods in sub class

The code using `Base::foo;` makes all the functions named `foo` (irrespective of their arguments) from base class directly accessible in the Derived class. Hence the functions named `foo` become overloaded in the scope of Derived class. In case of multiple `foo` functions in the base class, it is not possible to make only a few "`foo`" functions from the base class accessible in the derived class with "`Base::foo;`" declaration.

Since both versions of `foo` are available in the scope of Derived class, the compiler can choose specific function from them without any ambiguity here, and the call `foo(i)` is correctly resolved to `foo(int t)`.

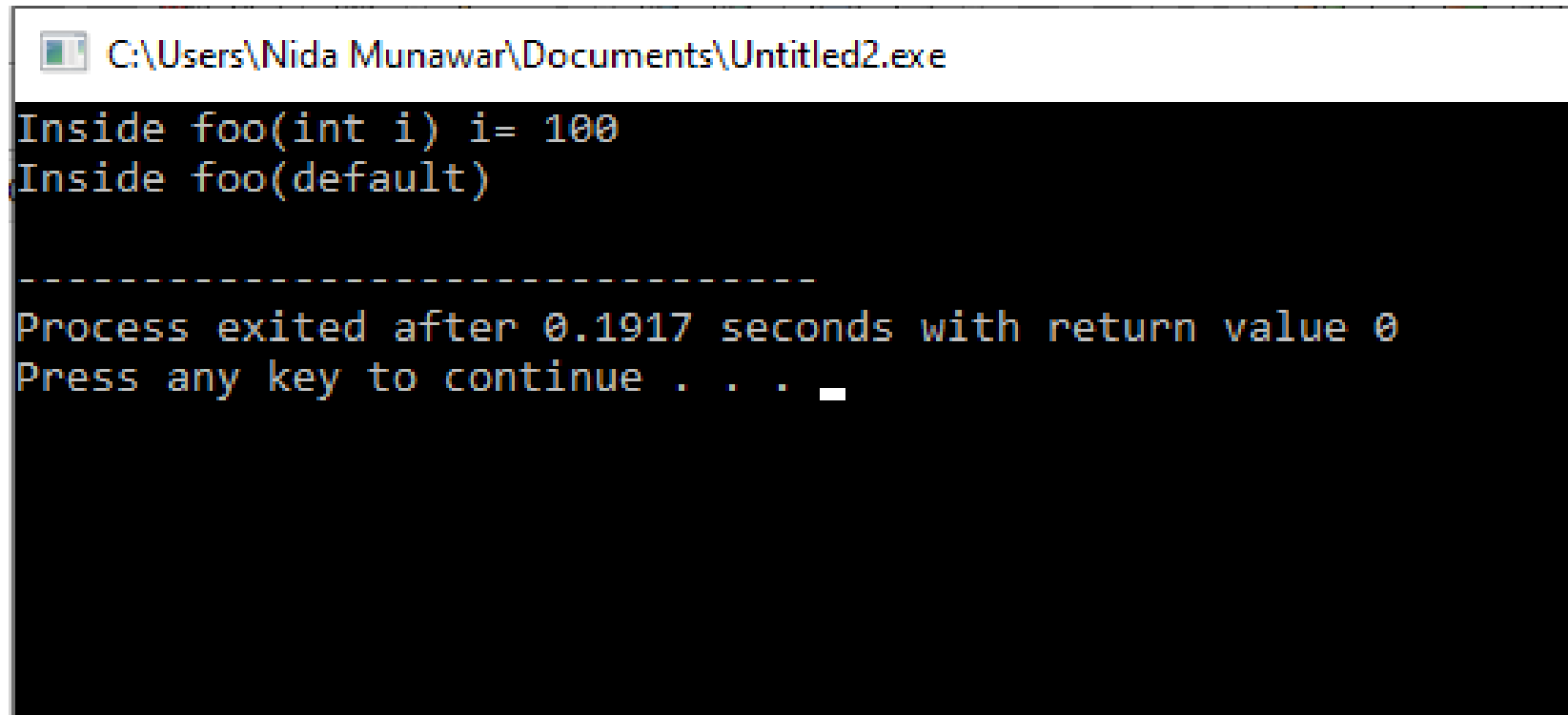
multiple foo functions in the base class

- class Base{
- public:
- void foo(int i) {
- cout << "Inside foo(int i) i= " << i << "\n"; }
- void foo() {
- cout << "Inside foo(default) " << "\n"; };
- class Derived : public Base{
- public:
- Base::foo;
- void foo(float f) {
- cout << "Inside foo(float f) f= " << f << "\n"; };

multiple foo functions in the base class

- `int main(){`
- `float f = 1.2f;`
- `int i = 100;`
- `Derived obj;`
- `obj.foo(i);`
- `obj.foo(); //no compiler error`
- `}`

multiple foo functions in the base class



```
C:\Users\Nida Munawar\Documents\Untitled2.exe
Inside foo(int i) i= 100
Inside foo(default)

-----
Process exited after 0.1917 seconds with return value 0
Press any key to continue . . .
```

Function Overloading VS Function Overriding

- 1.Inheritance:** Overriding of functions occurs when one class is inherited from another class. Overloading can occur without inheritance.
- 2.Function Signature:** Overloaded functions must differ in function signature ie either number of parameters or type of parameters should differ. In overriding, function signatures must be same.
- 3.Scope of functions:** Overridden functions are in different scopes; whereas overloaded functions are in same scope.
- 4.Behavior of functions:** Overriding is needed when derived class function has to do some added or different job than the base class function. Overloading is used to have same name functions which behave differently depending upon parameters passed to them.

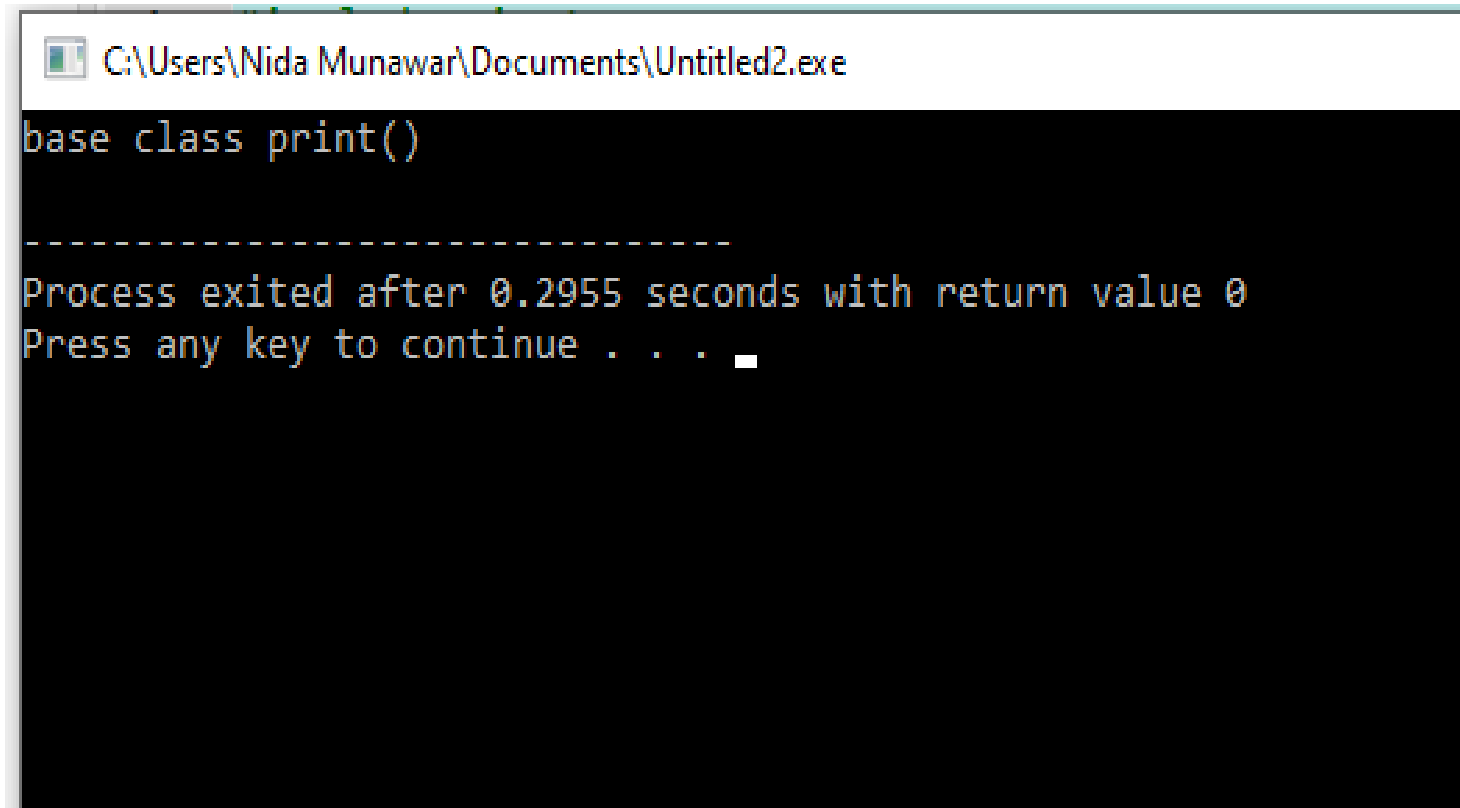
Week 10 Lecture 2 and 3

- **Runtime Polymorphism**

Compile-time(early binding)

```
class Base {  
    public:  
    void print() {  
        cout << "base class print()" << endl; }  
};  
class Derived : public Base {  
    public:  
    void print() {  
        cout << "derived class print()" << endl; }  
};  
int main() {  
    Derived derived1;  
    Base* base1 = &derived1; //up casting  
    base1->print();}
```

Compile-time(early binding)



```
C:\Users\Nida Munawar\Documents\Untitled2.exe  
base class print()  
-----  
Process exited after 0.2955 seconds with return value 0  
Press any key to continue . . .
```

Problem

- we aimed a base-class pointer at a derived-class object, then invoked member function print through that pointer. but the type of the handle(pointer) determines which class's functionality to invoke. In that case, the Base class pointer invoked the member function print on the derived-class object, even though the pointer was aimed at a derived-class object that has its own customized print function.
- **Solution**
- *With virtual functions, the type of the object being pointed to, not the type of the handle, determines which version of a virtual function to invoke.*

virtual function

- A virtual function is a member function that you expect to be redefined in derived classes. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.
- Virtual functions ensure that the correct function is called for an object, regardless of the expression used to make the function call.

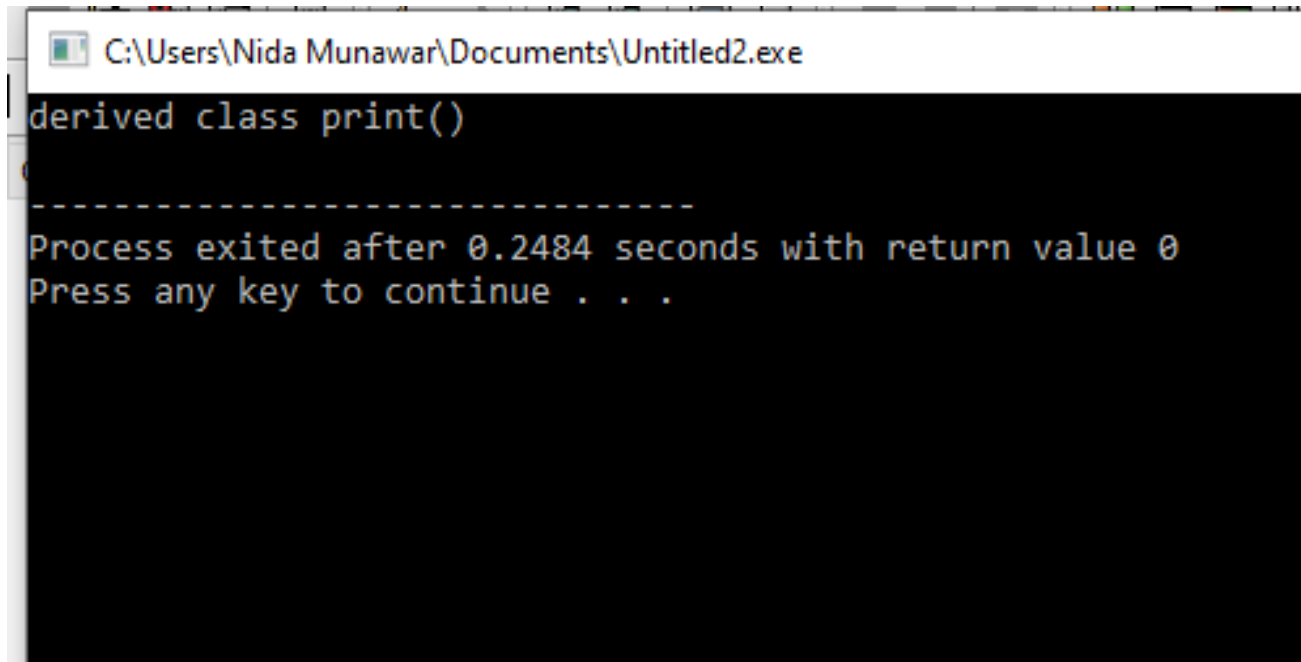
virtual function

- They are mainly used to achieve Runtime polymorphism
- Functions are declared with a **virtual** keyword in base class.
- The resolving of function call is done at Run-time.

virtual function

```
class Base {  
    public:  
        virtual void print() {  
            cout << "base class print()" << endl; }  
};  
class Derived : public Base {  
    public:  
        void print() {  
            cout << "derived class print()" << endl; }  
};  
int main() {  
    Derived derived1;  
    Base* base1 = &derived1;  
    base1->print();  
}
```

virtual function



```
C:\Users\Nida Munawar\Documents\Untitled2.exe  
derived class print()  
-----  
Process exited after 0.2484 seconds with return value 0  
Press any key to continue . . .
```


- Runtime polymorphism is achieved only through a pointer (or reference) of base class type
- Late binding(Runtime) is done in accordance with the content of pointer (i.e. location pointed to by pointer) and Early binding(Compile time) is done according to the type of pointer

When calling a function using pointers or references, the following rules apply:

1. Functions in derived classes override virtual functions in base classes only if their type is the same. A function in a derived class cannot differ from a virtual function in a base class in its return type only; the argument list must differ as well.
2. A call to a virtual function is resolved according to the underlying type of object for which it is called.
3. A call to a nonvirtual function is resolved according to the type of the pointer or reference.
4. The virtual keyword can be used when declaring overriding functions in a derived class, but it is unnecessary; overrides of virtual functions are always virtual.

Rule 4

```
class Base {
```

- public:
- virtual void print() {
- cout << "base class print()" << endl; };
- class Derived : public Base {
- public:
- virtual void print() { // virtual keyword with derived version can be used
- cout << "derived class print()" << endl;};
- int main() {
- Derived derived1;
- Base* base1 = &derived1;
- base1->print();}

 C:\Users\Nida\Documents\Untitled1.exe

```
derived class print()
```

```
-----  
Process exited after 5.603 seconds with return value 0
```

```
Press any key to continue . . .
```

Working of virtual functions(concept of VTABLE and VPTR)

- If a class contains a virtual function, then compiler itself does two things:

1.vtable: A table of function pointers, maintained per class.

It is a table that contains the memory addresses of all virtual functions(only VFs) of a class in the order in which they are declared in a class. This table is used to resolve function calls in dynamic/late binding manner. Every class that has virtual function will get its own Vtable.

Working of virtual functions(concept of VTABLE and VPTR)

2. *vptr*: A pointer to vtable, maintained per object instance.

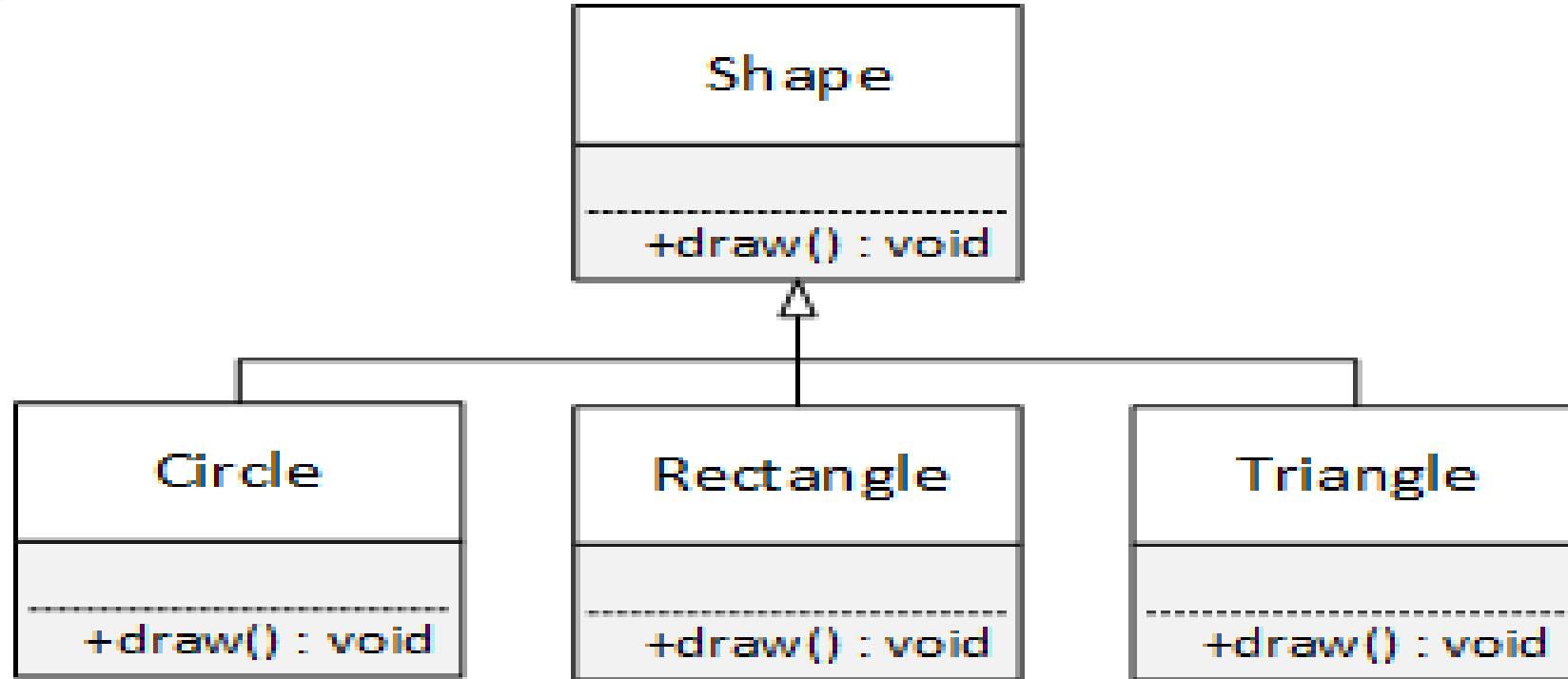
- After creating Vtable address of that table gets stored inside a pointer i.e. VPTR (Virtual Pointer). When you create an object of a class which contains virtual function then a hidden pointer gets created automatically in that object by the compiler. That pointer points to a virtual table of that particular class.

- Compiler adds additional code at two places to maintain and use *vp*tr.

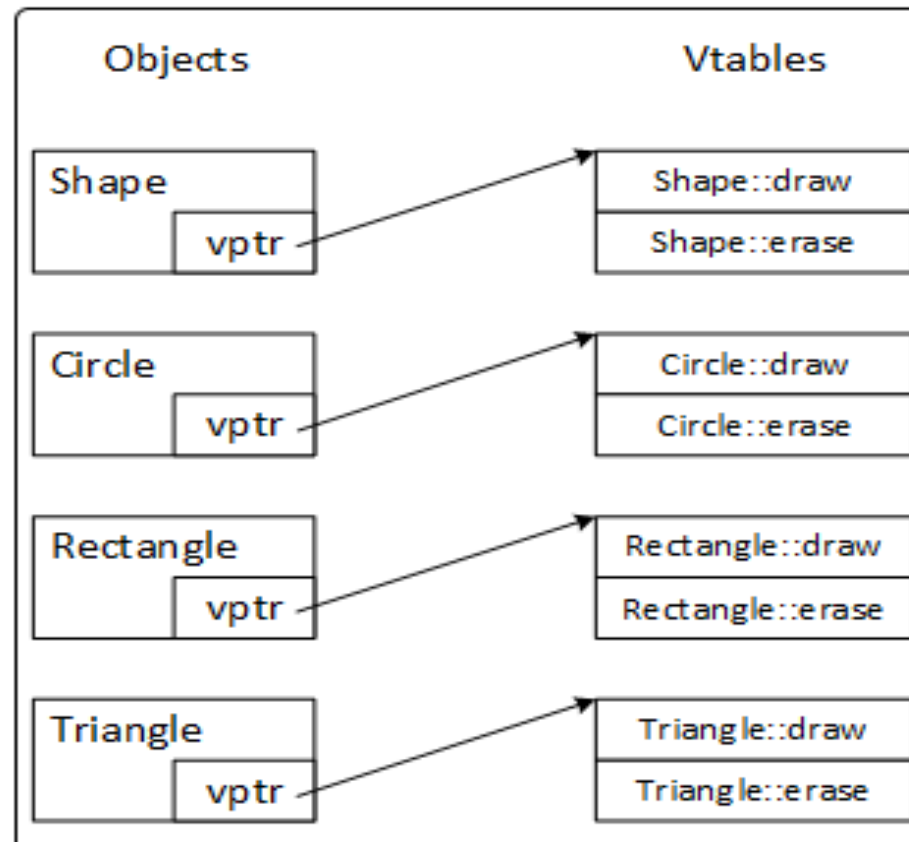
1) Code in every constructor. This code sets the *vp*tr of the object being created. This code sets *vp*tr to point to the *v*table of the class.

2) Code with polymorphic function call (e.g. `base1->print()` in above code). Wherever a polymorphic call is made, the compiler inserts code to first look for *vp*tr using base class pointer or reference (In the above example, since pointed or referred object is of derived type, *vp*tr of derived class is accessed). Once *vp*tr is fetched, *v*table of derived class can be accessed. Using *v*table, address of derived class function *print()* is accessed and called.

Example (concept of VTABLE and VPTR)



Example (concept of VTABLE and VPTR)



Example (concept of VTABLE and VPTR)

- Every polymorphic class has a table of pointers called the vtable that points to the class's virtual functions.
- Every polymorphic object contains a pointer called the vptr that points to its class's vtable.

Example (concept of VTABLE and VPTR)

- The vptr and the vtable are only used to locate virtual functions. When all of the prerequisites for polymorphism are in place and a virtual function is called, the correct function is located and dispatched (i.e., called) by a specific sequence of steps. Assume that we have a Circle object named *c* created, and further assume that the *draw* function is a virtual function.
- Circle c;
- Shape * s = &c;
- s->draw();

The steps that the compiler follows to call the correct function is as follows:

- 1.The address stored in c's vptr is retrieved
- 2.The address is followed to the Circle vtable (this address was stored in c's vptr by the Circle constructor)
- 3.The Circle vtable is searched for a function named draw
- 4.The function to which the draw entry in the vtable points is called

polymorphic does have some overhead:

- It requires enough space in *every* polymorphic object to store a pointer (four to eight bytes)
- It requires enough space to store the address of every virtual function in a program (i.e., there is a pointer entry in every vtable of every class in an inheritance hierarchy for each virtual function)
- There are two pointer dereference operations for every polymorphic function call: one to find the vtable and one to find the function
- There is time required to search the vtable for the correct virtual function

Task

```
Class A{
```

```
Virtual void foo1();
```

```
Virtual void foo2();
```

```
Virtual void foo3();
```

```
    void foo();};
```

```
Class B : public A{
```

```
void foo1();
```

```
void foo2(2);
```

```
    void foo();}
```

Derived-Class Member-Function Calls via Base-Class Pointers

- Off a base-class pointer, the compiler allows us to invoke only base-class member functions. (**object slicing we will discuss it in RTTI**)
- Thus, if a base-class pointer is aimed at a derived-class object, and an attempt is made to access a ***derived-class-only member function***, a compilation error will occur.
- The compiler generates errors on these calls, because they're not made to member functions of base-class

Derived-Class Member-Function Calls via Base-Class Pointers

- `class Base {`
- `public:`
- `virtual void print() {`
- `cout << "base class print()" << endl; };`
- `class Derived : public Base {`
- `public:`
- `void print() {`
- `cout << "derived class print()" << endl;}`
- `void show() {`
- `cout << "only derived class member function" << endl;};`
- `int main() {`
- `Derived derived1;`
- `Base* base1 = &derived1;`
- `base1->print();`
- `base1->show();}`

Derived-Class Member-Function Calls via Base-Class Pointers

Message

In function 'int main()':

[Error] 'class Base' has no member named 'show'

- Can I override static function C++?

Can I override static function C++?

- **Static methods cannot be overridden** because method overriding only occurs in the context of dynamic (i.e. runtime) lookup of methods. Static methods (by their name) are looked up statically (i.e. at compile-time).

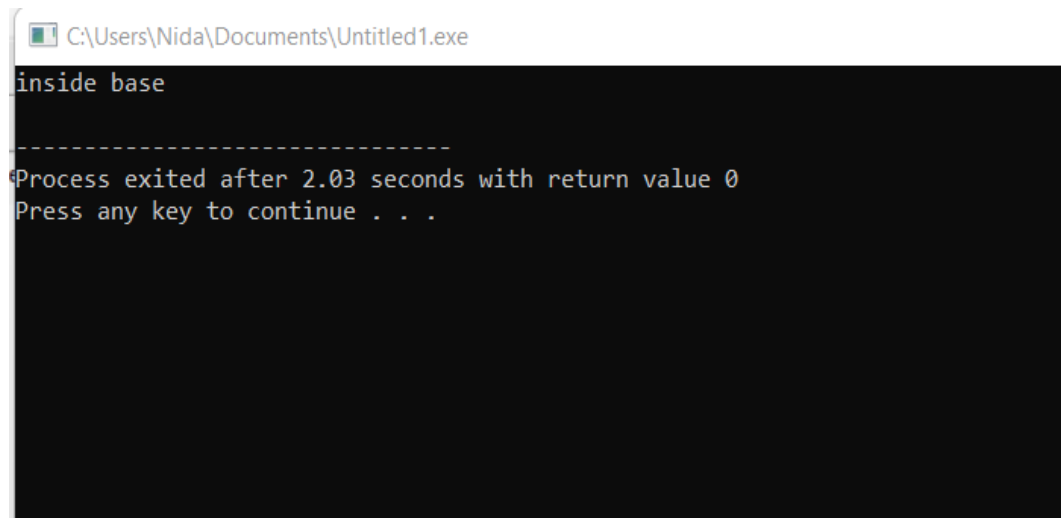
Early binding

```
#include <iostream>
using namespace std;
class base{
    public:
        static void func(){
            cout << "inside base"<< endl;
        }
};

class child:public base{
    public:
        void func(){
            cout << "inside child"<< endl;
        }
};

int main(){
    base* b ;
    child c;
    b = &c;
    b -> func();
}
```

Early binding



```
C:\Users\Nida\Documents\Untitled1.exe
inside base
-----
Process exited after 2.03 seconds with return value 0
Press any key to continue . . .
```

Static in run time polymorphism

- `#include <iostream>`
- `using namespace std;`
- `class base{`
- `public:`
- `virtual static void func(){`
- `cout << "inside base"<< endl;`
- `};`
- `class child:public base{`
- `public:`
- `void func(){`
- `cout << "inside child"<< endl;`
- `};`
- `int main(){`
- `base* b ;`
- `child c;`
- `b = &c;`
- `b -> func();`
- `}`

 Compiler (1)  Resources  Compile Log  Debug  Find Results  Close			
Line	Col	File	Message
5	28	C:\Users\Nida\Documents\Untitled1.cpp	[Error] member 'func' cannot be declared both virtual and static

Important point

- A derived class virtual function is only considered overridden if its signature and return type must match exactly

A derived class virtual function is only considered overridden if its signature and return type must match exactly

- `class Base {`
- `public:`
- `virtual void print() {`
- `cout << "base class print()" << endl; };`
- `class Derived : public Base {`
- `public:`
- `int print() {`
- `cout << "derived class print()" << endl;`
- `return 1;}`
- `};`
- `int main() {`
- `Derived derived1;`
- `Base* base1 = &derived1 ; //up casting`
- `base1->print();`
- `}`

 Compiler (2)  Resources  Compile Log  Debug  Find Results  Close			
Line	Col	File	Message
9	9	C:\Users\Nida\Documents\Untitled2.cpp	[Error] conflicting return type specified for 'virtual int Derived::print()'
5	18	C:\Users\Nida\Documents\Untitled2.cpp	[Error] overriding 'virtual void Base::print()'

- can we override const method with non const in
C++

It is neither overloading nor overriding.
Rather, it is *hiding*.

- `const` is part of the signature, and **leaving it off changes the signature, and thus hides the method rather than overrides it.** "

- class Base {
- public:
- virtual void print() const{
- cout << "base class print()" << endl; };
- class Derived : public Base {
- public:
- void print() {
- cout << "derived class print()" << endl;
- }
- };
- int main() {
- Derived derived1;
- Base* base1 = &derived1 ; //up casting
- base1->print();
- }

 C:\Users\Nida\Documents\Untitled2.exe

```
base class print()
```

```
-----
```

```
Process exited after 1.375 seconds with return value 0
```

```
Press any key to continue . . .
```

How to prevent class inheritance and method overriding in C++

- **Use of final specifier in C++ 11:**

uses final specifier after the function parameter list to prevent method overriding

- `class Base {`
- `public:`
- `virtual void print() final{`
- `cout << "base class print()" << endl; };`
- `class Derived : public Base {`
- `public:`
- `void print() {`
- `cout << "derived class print()" << endl; };`
- `int main() {`
- `Derived derived1;`
- `Base* base1 = &derived1 ; //up casting`
- `base1->print();}`

 Compiler (3)  Resources  Compile Log  Debug  Find Results  Close			
Line	Col	File	Message
5	25	C:\Users\Nida\Documents\Untitled2.cpp	[Warning] override controls (override/final) only available with -std=c++11 or -std=gnu++11
9	10	C:\Users\Nida\Documents\Untitled2.cpp	[Error] virtual function 'virtual void Derived::print()'
5	19	C:\Users\Nida\Documents\Untitled2.cpp	[Error] overriding final function 'virtual void Base::print()'

final specifier is used after the class name to prevent inheritance

- `#include <iostream>`
- `using namespace std;`
- `class Base final {`
- `};`
- `class Derived : public Base {`
- `};`
- `int main() {`
- `Derived derived1;`
- `}`

 Compiler (2)  Resources  Compile Log  Debug  Find Results  Close

Line	Col	File	Message
3	7	C:\Users\Nida\Documents\Untitled2.cpp	[Warning] override controls (override/final) only available with -std=c++11 or -std=gnu++11
5	7	C:\Users\Nida\Documents\Untitled2.cpp	[Error] cannot derive from 'final' base 'Base' in derived type 'Derived'